



# **Trasformare un'idea in un videogioco**

Un viaggio pratico nel game development con Godot

Speaker Lorenzo Mazzola



# Cos'è Godot

---

## Game Engine Completo

Editor che comprende, renderer grafico, fisica, UI, animazioni, audio, export e scripting in un unico ambiente.

## Workflow dedicati per 2D e 3D

Mette a disposizione workflow dedicati per 2D e 3D invece di trattare il 2D come una variante del 3D.

## Open source

Codice disponibile, nessuna royalty e libertà di modifica/distribuzione del software.

1. Godot è pensato per prototipare velocemente e poi scalare l'idea in modo semplice.
2. L'idea di base per costruire un progetto è quella di progettare scene riutilizzabili formate da **nodi** ed aggiungere il comportamento tramite **script** e **segnali**.

# Come funziona l'editor

## Scene Tree

Gerarchia dei nodi: “chi contiene chi” e l’ordinamento, ovvero “chi viene renderizzato prima”

## FileSystem

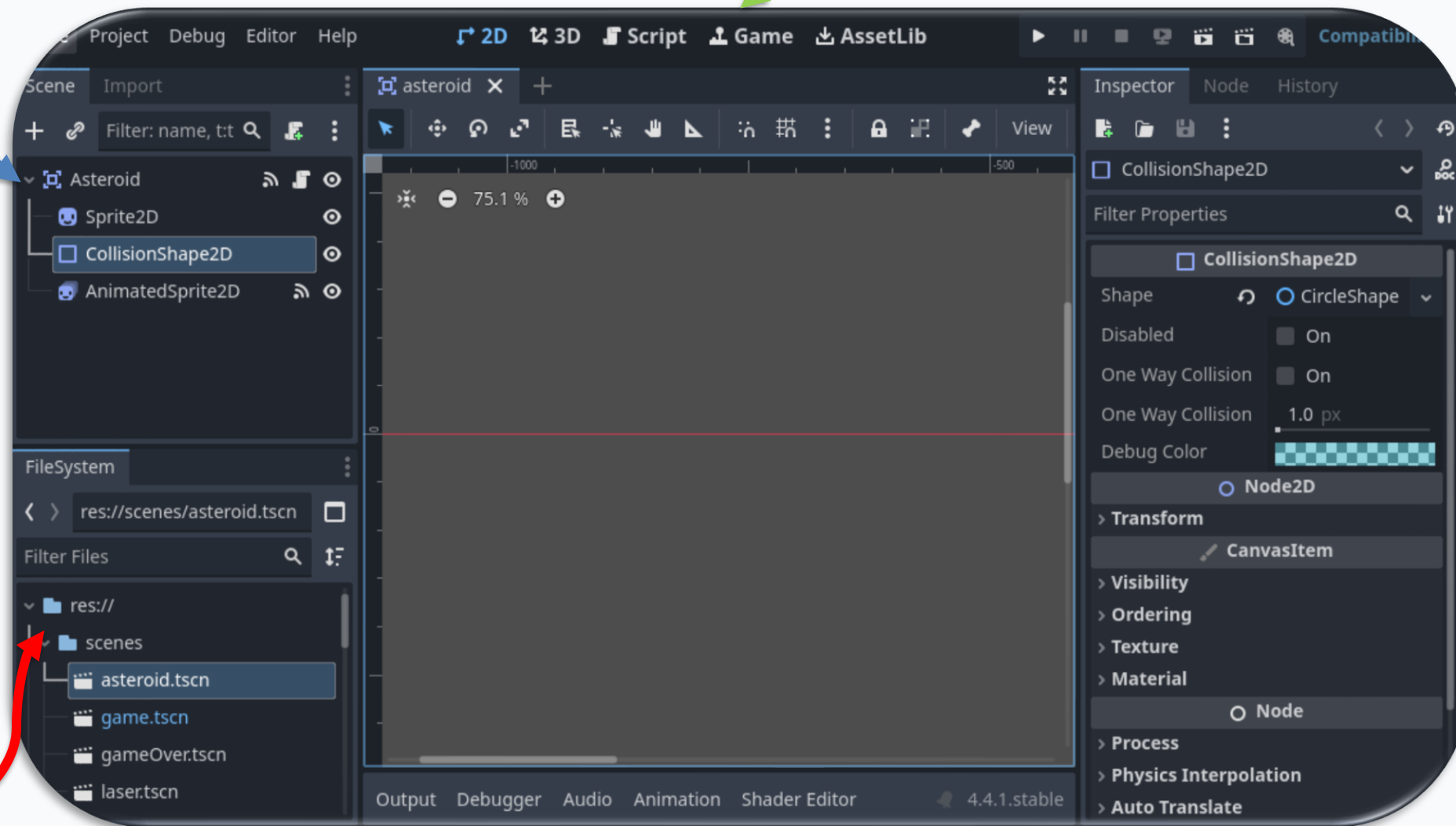
Asset, scene .tscn, script, immagini, audio, risorse.

## Viewport

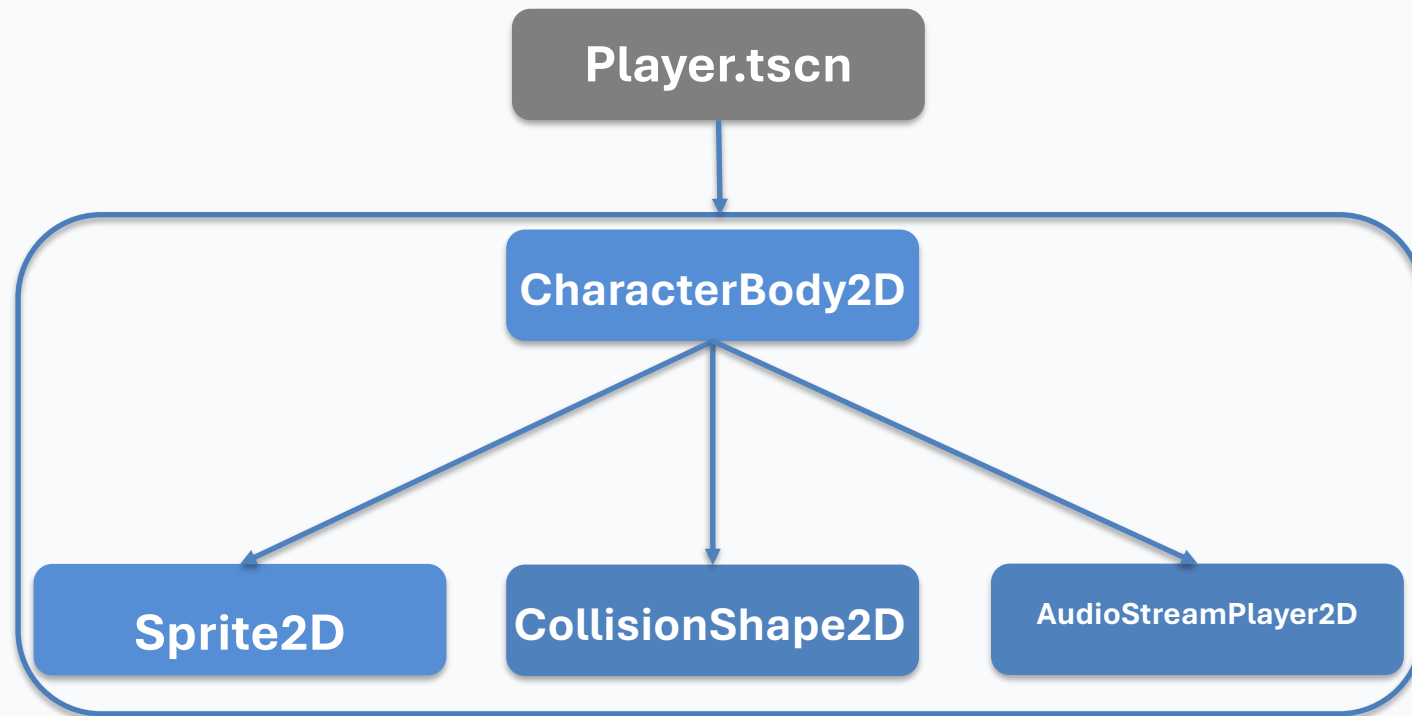
Area di visualizzazione della scena 2D/3D: elementi, forme, camere, collisioni, UI.

## Inspector

Proprietà del nodo selezionato. Ad esempio trasformazioni, visibilità.



# La base di Godot - I nodi

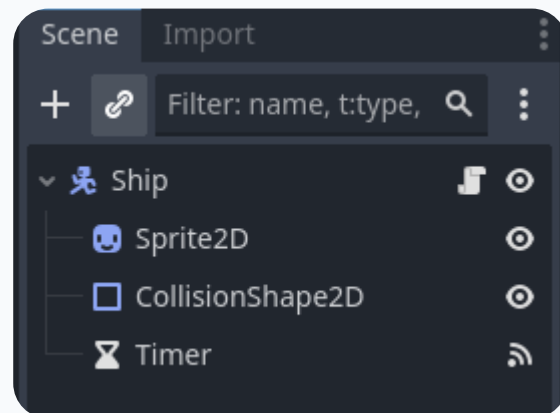


## Scena

Un albero di nodi salvato come file. Può essere qualsiasi oggetto del gioco: livello, personaggio, UI, proiettile, nemico.

## Nodo

Unità minima con uno scopo specifico: disegna, riproduce un audio, gestisce le collisioni o un input, o semplicemente viene usato come contenitore altri nodi.



Una scena può essere riusata dentro un'altra scena.  
È il meccanismo base per costruire progetti modulari.

# Nodi: Alcune tipologie

## Node / Node2D / Node3D

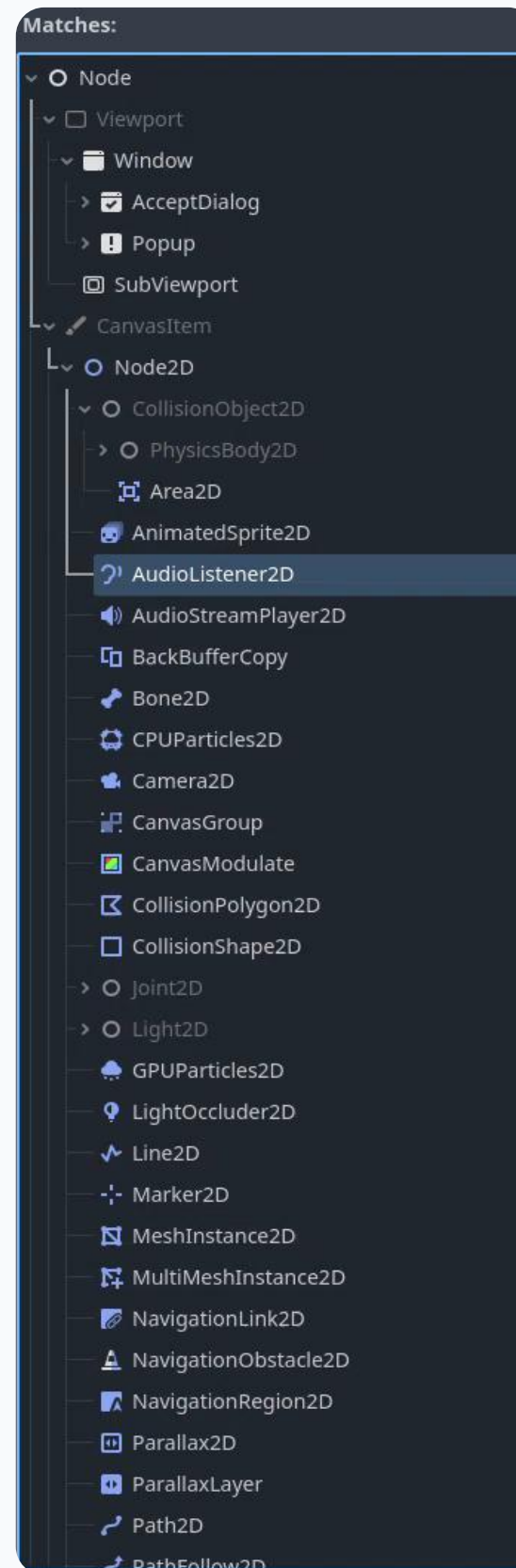
Nodo di base per gestire le logiche più semplici di Godot: non ha nessuna specializzazione. Radice di tutti gli altri nodi.

## Control

Nodi specializzati per disegnare la UI: bottoni, label, pannelli, menu.

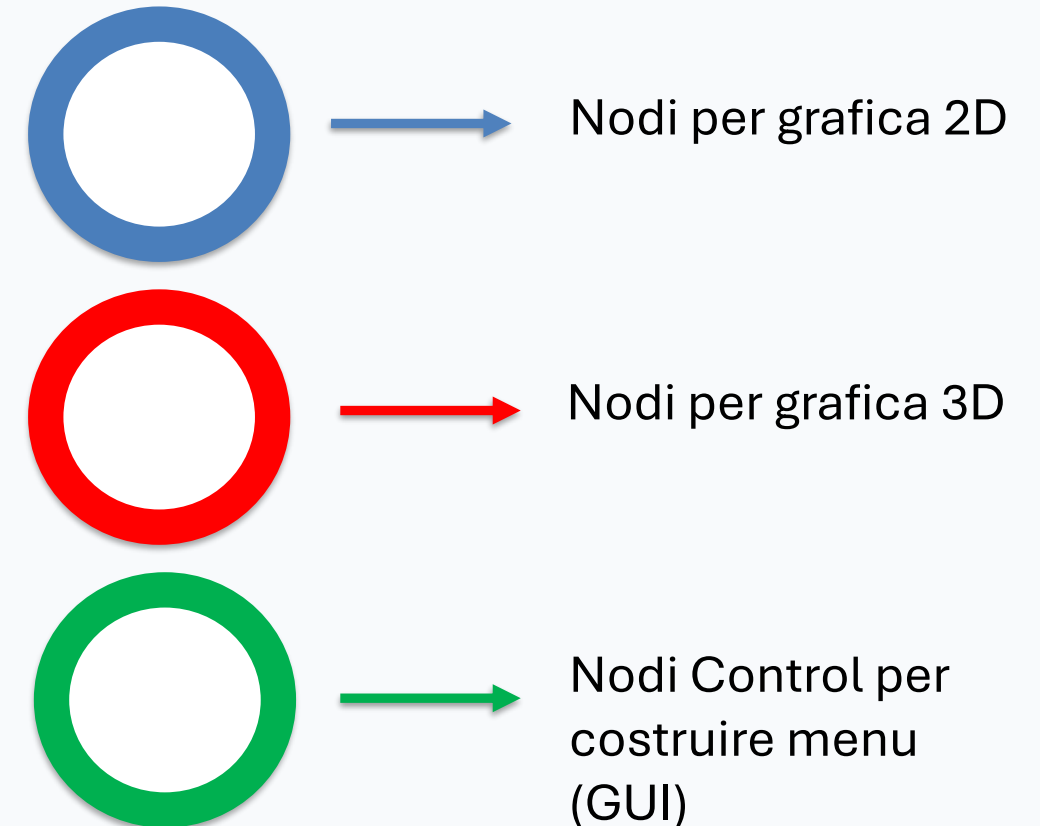
## Area2D/3D + CollisionShape2D/3D

Nodo per gestire aree e, insieme al CollisionShape2D/3D anche le collisioni con altri nodi/mouse.



## CharacterBody2D/3D

Nodo che mette a disposizione nativamente metodi per controllare movimento/collisione.



# Script in Godot - GDScript

```
extends Area2D

@export var speed: int = 300

@onready var sprite = $Sprite2D

var direction = Vector2(0, -1)

func _ready() -> void:
    add_to_group("LASER")

func _process(delta: float) -> void:
    position += direction * speed * delta
```

## **\_ready()**

Chiamato quando il nodo entra nella scena ed è pronto: setup iniziale.

## **\_process(delta)**

Chiamato ogni frame, ideale per aggiornare la UI, nodi e registrare l'input del giocatore

## **\_physics\_process(delta)**

Specializzato nell'aggiornamento del motore fisico, movimento e collisioni.

# Signals

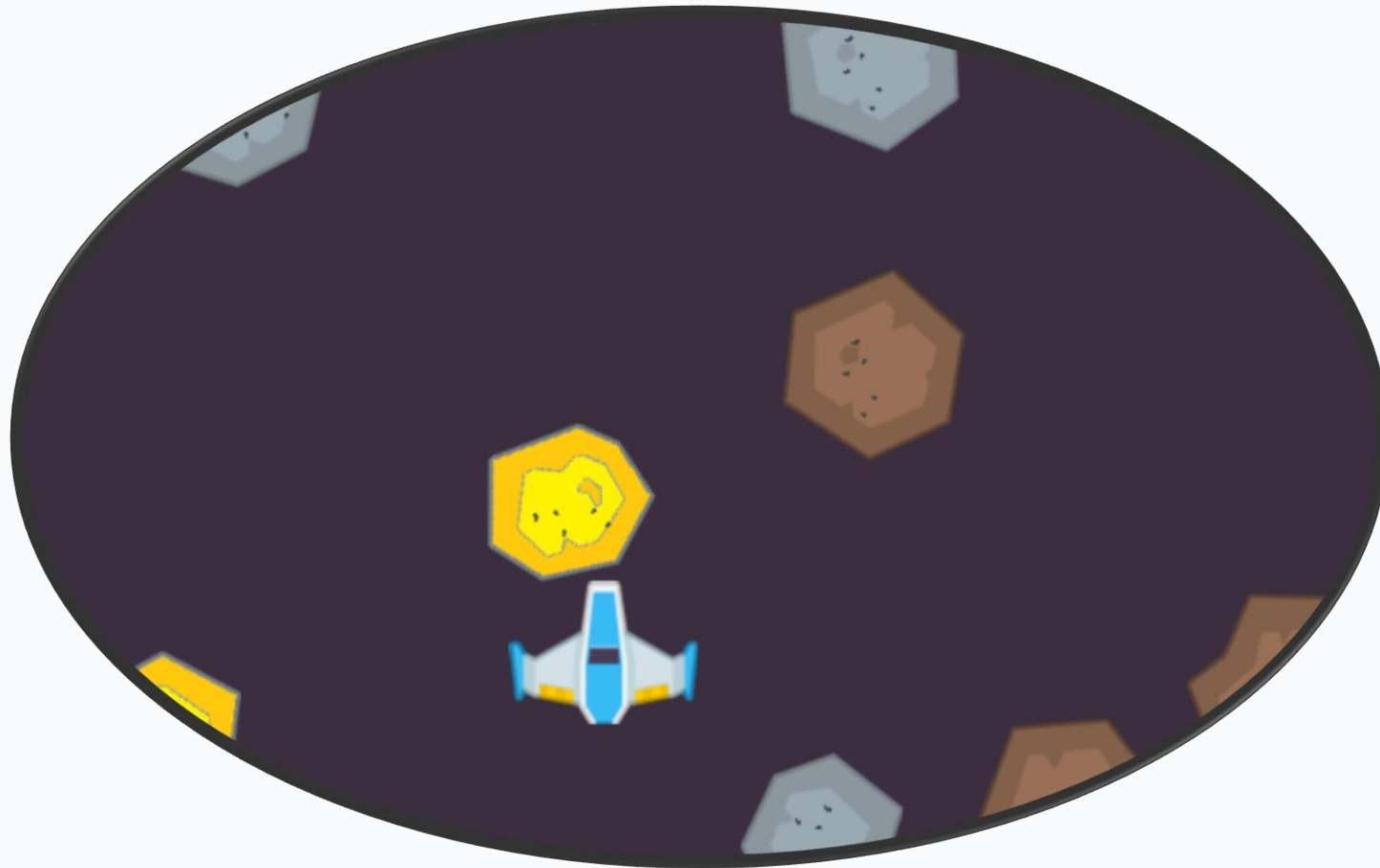
Il nodo Asteroide chiama direttamente il GameManager (Autoload) che aggiorna gli HP del giocatore

La UI è sempre in ascolto di un segnale custom di GameManager  
`GameManager.connect("update_score", _on_update_score)`

Il nodo Area2D  
Asteroide identifica  
una collisione con il  
giocatore

Game Manager riceve  
aggiornamento e  
notifica UI tramite  
segnale

UI



## Regola pratica

“Call down, signal up”: il padre può comandare direttamente i figli, ma un figlio dovrebbe notificare verso l'esterno con un segnale (che sia il padre o uno script globale).

## Perché usarli

- Chi emette non deve conoscere chi ascolta
- Perfetti per UI, collisioni, notifiche verso altri nodi
- Rendono le scene più riutilizzabili e testabili

# Global Script/Autoload

## A cosa servono

Contenere e gestire variabili globali, metodi condivisi, configurazioni.

## Come funzionano

Lo aggiungi in Project Settings → Autoload: Godot lo istanzia all'avvio.

## Come si usano

Invochi l'autoload per nome e di seguito un metodo o una variabile: `GameManager.score += 1`.

**Giocatore raccoglie  
una moneta**

**GameManager.gd**

Nello script del giocatore andremo a richiamare il GameManager:

```
GameManager.score+=1  
Oppure  
GameManager.addScore(1)
```

# Inheritance e Composition in Godot

## Inheritance

Creo una scena base o una classe base e ne estendo delle varianti.

Esempio: Enemy → FlyingEnemy, BossEnemy.

Utile quando condividono metodi base e solo alcuni devono essere modificati o aggiunti.

### Usa inheritance quando

- Le differenze sono poche tra la classe di partenza e quella nuova, non devo quindi stravolgerne il comportamento.
- Mi serve fare solo override di metodi comuni e non creare intere classi nuove per implementare il comportamento che ci serve.

## Composition

Aggiungo componenti riusabili in scene completamente diverse tra di loro.

Esempio: HealthComponent, HitboxComponent.

Utile quando tanti oggetti diversi condividono una specifica funzionalità

### Usa composition quando

- Vuoi creare feature riutilizzabili su giocatore, nemici, oggetti.
- Hai tanti elementi diversi ma che condividono stesse caratteristiche. Esempio: Giocatore, nemici e oggetti hanno tutti dei punti vita, quindi è consigliabile utilizzare un component Health per tutti.

# Confronto rispetto ad altri engine

## Godot

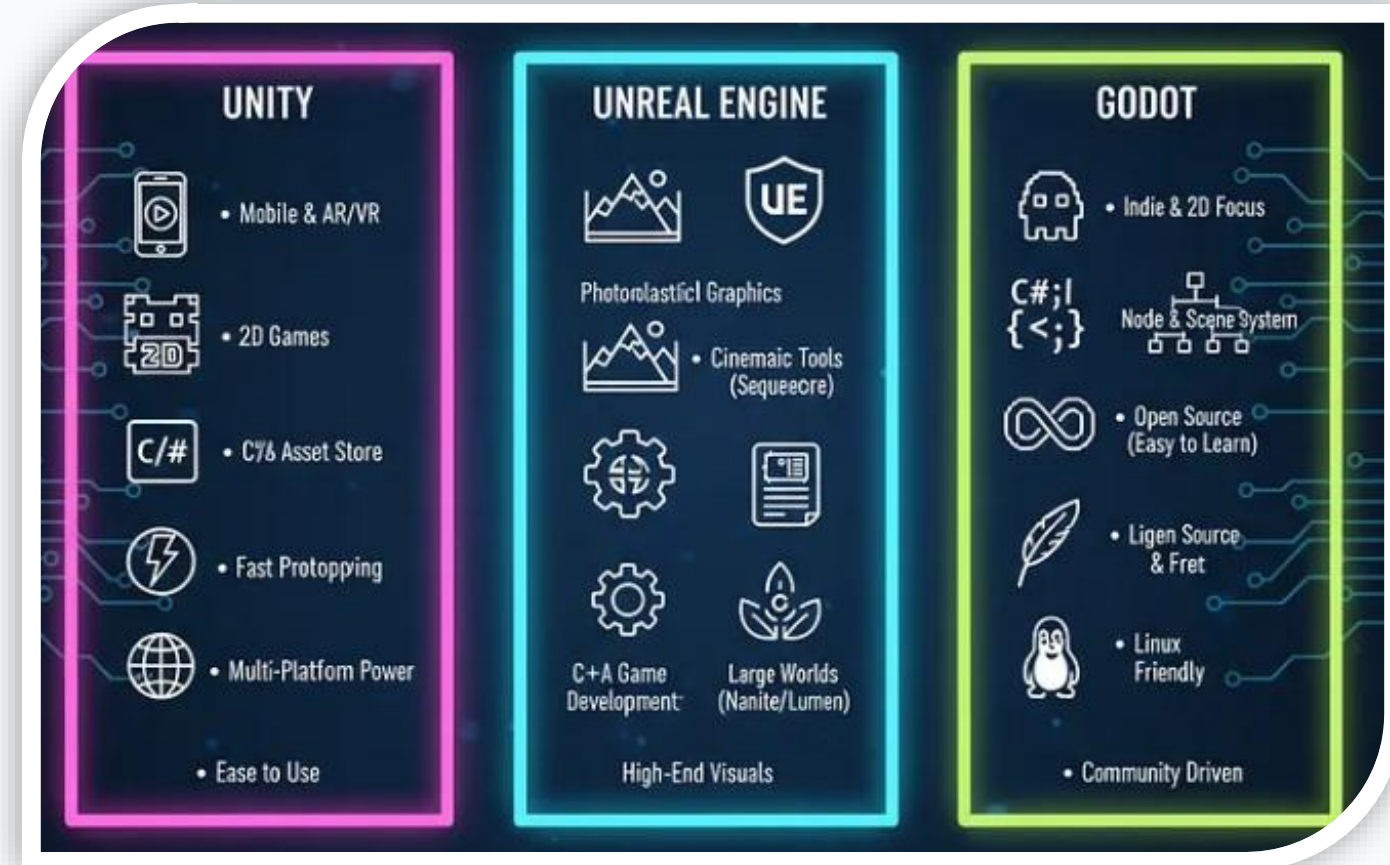
Scene/nodi sono elementi riutilizzabili, GDScript linguaggio molto rapido da compilare, sotto licenza MIT, editor molto leggero, community in rapida espansione e per sempre gratuito.

## Unity

GameObject + Component, utilizza C# come linguaggio, asset store molto fornito, ma alcune features pro sono a pagamento.

## Unreal

Actor/Component, rendering per giochi AAA, C++ come linguaggio, 5% in royalty fees di tutti gli incassi di giochi.



# Demo live - MD Space Game!

---

## Obiettivo

Muovere una navicella, evitare gli asteroidi, fare punti distruggendoli e aggiornare il punteggio nella UI.

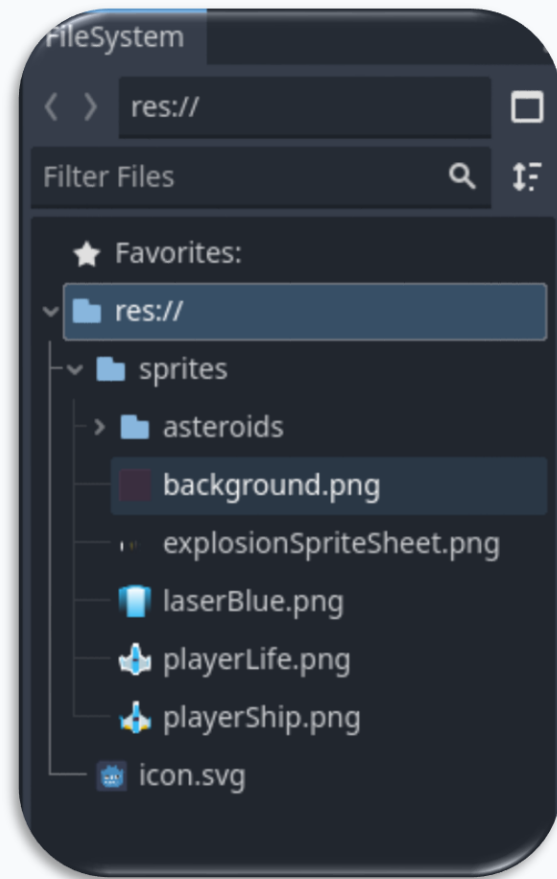
## Concetti mostrati

Scene, nodi 2D, collisioni, script e segnali.

## Project settings

- Screen size: 1280x720.  
Project -> Project Settings -> Viewport Width e Height
- Godot Version: 4.4.1

# Demo live - Asset

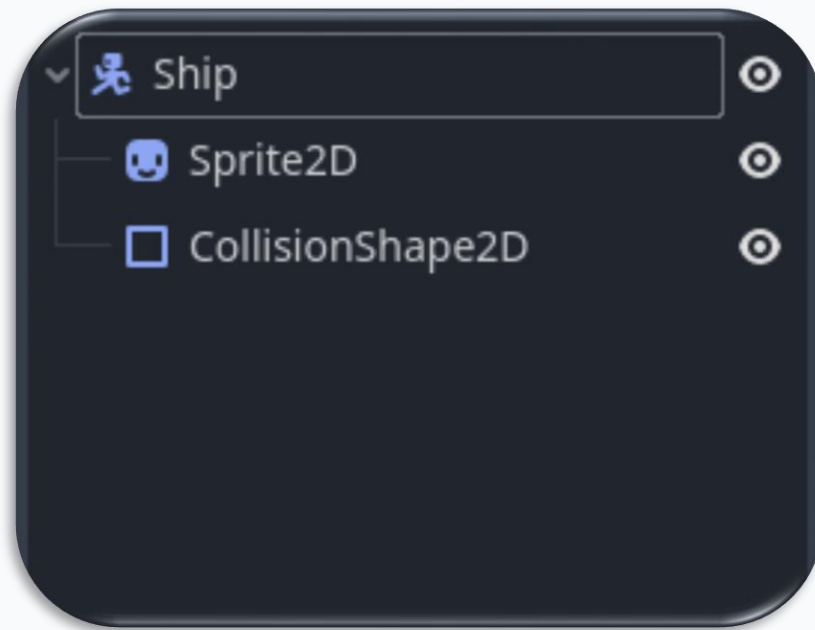


Importiamo gli asset che ci serviranno per il progetto all'interno della cartella root res://



# Demo live - Ship Scene

---



Nodo di tipo `CharacterBody2D` per poter sfruttare i metodi già implementati per la gestione del movimento.

Come figli del nodo `Ship`, aggiungiamo:

- `Sprite2D` per disegnare l'immagine della navicella.
- `CollisionShape2D` per gestire le collisioni con l'ambiente (asteroidi o muri).

# Demo live - Ship.gd

```
extends CharacterBody2D
```

```
@export var speed = 400
```

```
#Ricava il vettore con le direzioni premute rispetto ai tasti e moltiplica per la velocità configurata
```

```
func get_input():
```

```
    var input_direction = Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
```

```
    #velocity è una variabile ereditata da CharacterBody2D e possiamo usarla senza dichiararla
```

```
    velocity = input_direction * speed
```

```
#Nel ciclo principale calcolo velocity nella get_input e poi chiamo move_and_slide
```

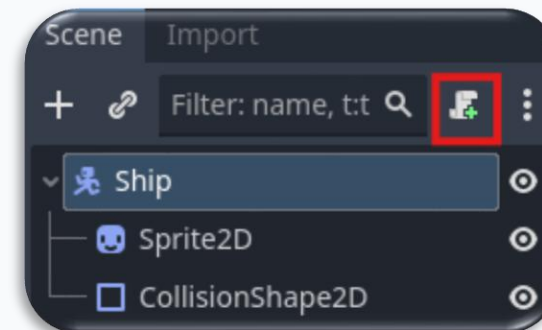
```
#così che il movimento e le collisioni vengano calcolate durante il frame
```

```
func _process(delta):
```

```
    get_input()
```

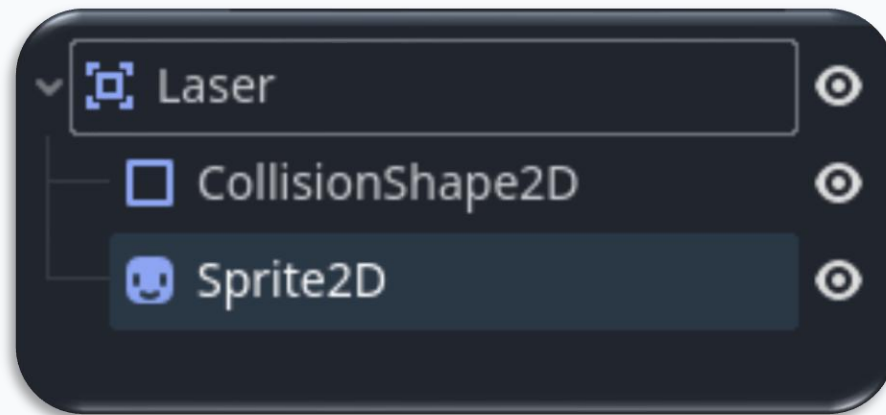
```
    move_and_slide()
```

**Note:** Colleghiamo al nodo Ship un nuovo script, salvandolo in `res://scripts`.



# Demo live - Laser Scene

---



Nodo di tipo Area2D dato che servirà solo per gestire le collisioni.

Come figli del nodo Laser, aggiungiamo:

- Sprite2D per disegnare l'immagine del laser.
- CollisionShape2D per gestire le collisioni con gli asteroidi.

# Demo live - Laser.gd

---

```
extends Area2D
```

```
@export var speed: int = 300
```

```
@onready var sprite = $Sprite2D
```

```
var direction = Vector2(0, -1)
```

```
func _process(delta: float) -> void:  
    position += direction * speed * delta
```

# Demo live - Combiniamo Ship e Laser (Ship.gd)

```
extends CharacterBody2D
```

```
@export var speed = 400
```

```
@onready var fire_rate_timer = $Timer
var laser = preload("res://scenes/laser.tscn")
var can_shoot = true
```

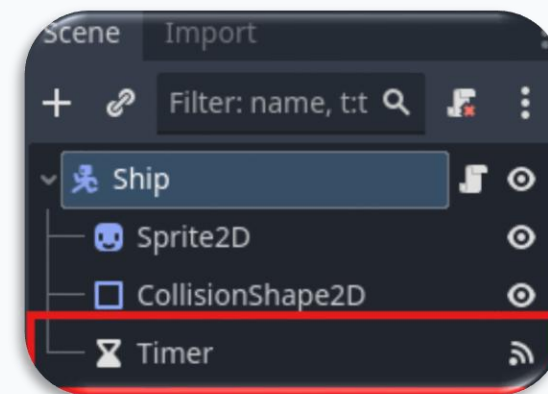
```
#Ricava il vettore con le direzioni premute rispetto ai tasti e moltiplica
#per la velocità configurata
func get_input():
    var input_direction = Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    #velocity è una variabile ereditata da CharacterBody2D e possiamo usarla
    #senza dichiararla
    velocity = input_direction * speed
```

```
#Nel ciclo principale calcolo velocity nella get_input e poi chiamo move_and_slide
#così che il movimento e le collisioni vengano calcolate durante il frame
func _process(delta):
    get_input()
    move_and_slide()
```

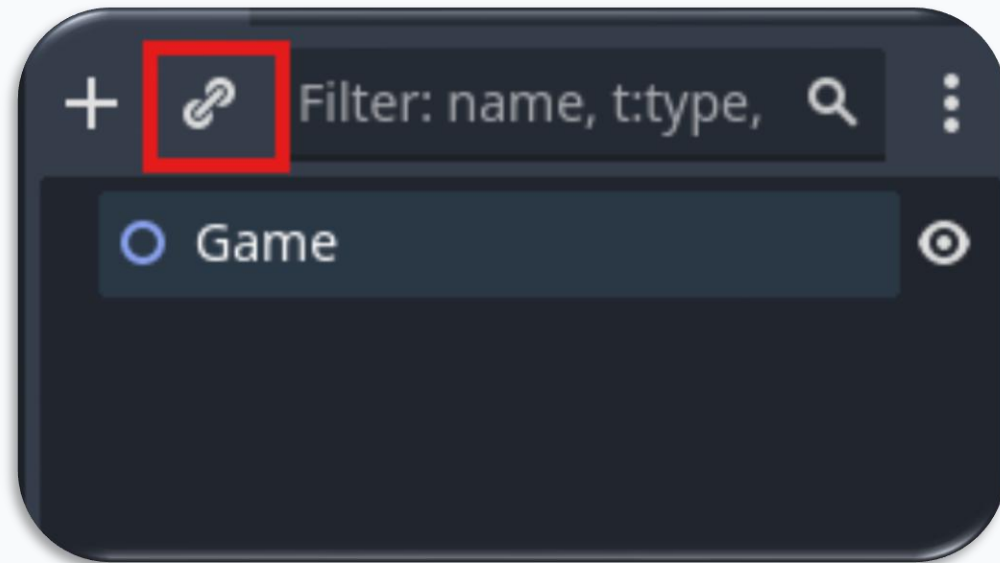
```
#_input ci permette di controllare che tasti sono premuti.
Se il tasto "SPAZIO" è
#premuto, allora spariamo il laser
func _input(event):
    if event is InputEventKey and event.pressed:
        if event.keycode == KEY_SPACE and can_shoot:
            can_shoot = false
            shoot_laser()
            fire_rate_timer.start()
```

```
#Metodo per istanziare la scena laser.tscn nell'albero delle
scene.
func shoot_laser() -> void:
    var laserScene = laser.instantiate()
    get_tree().current_scene.add_child(laserScene)
    laserScene.position = position
```

```
#Segnale di Timer - Da collegare
func _on_timer_timeout() -> void:
    can_shoot = true
```

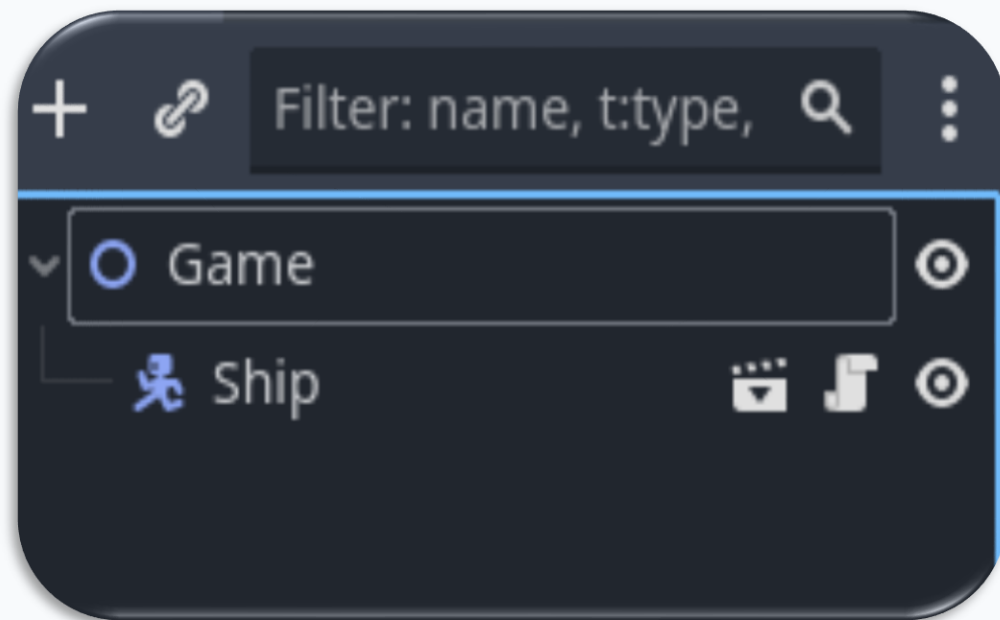


# Demo live - Game Scene



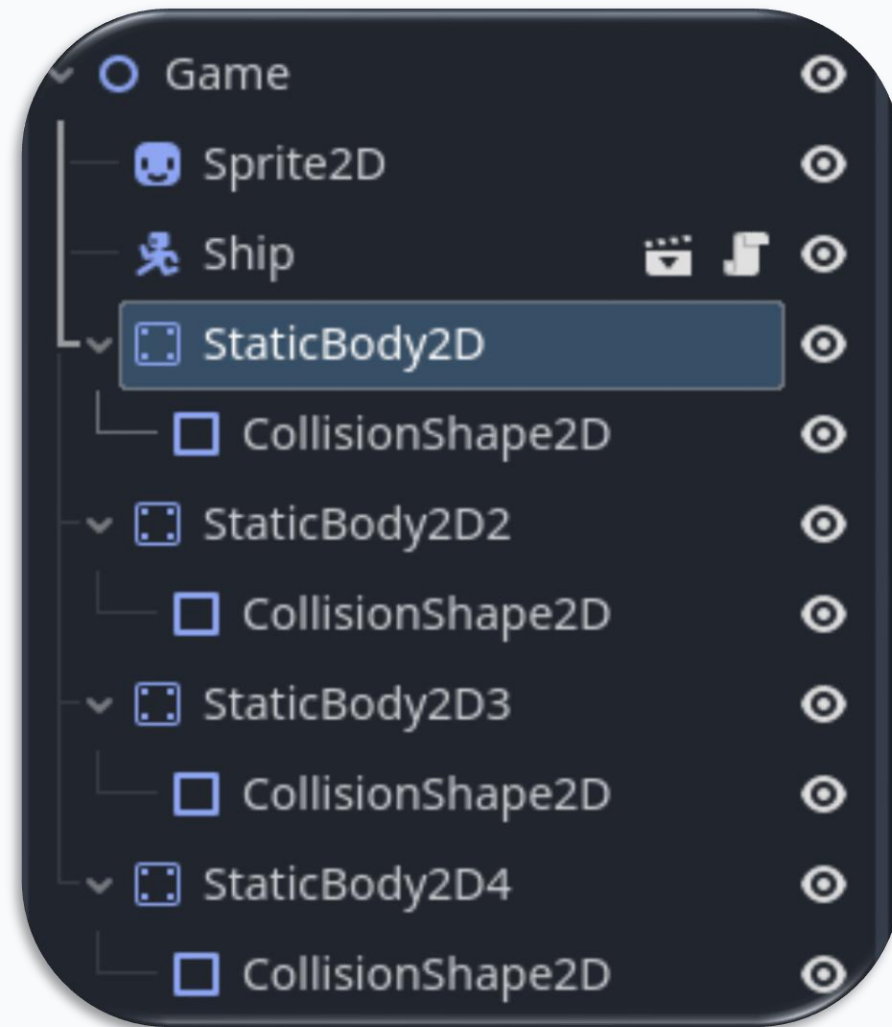
Nodo di tipo Node2D che servirà come contenitore per tutti gli elementi del gioco.

Con l'icona evidenziata possiamo aggiungere scene già create in precedenza dentro il nodo Game.

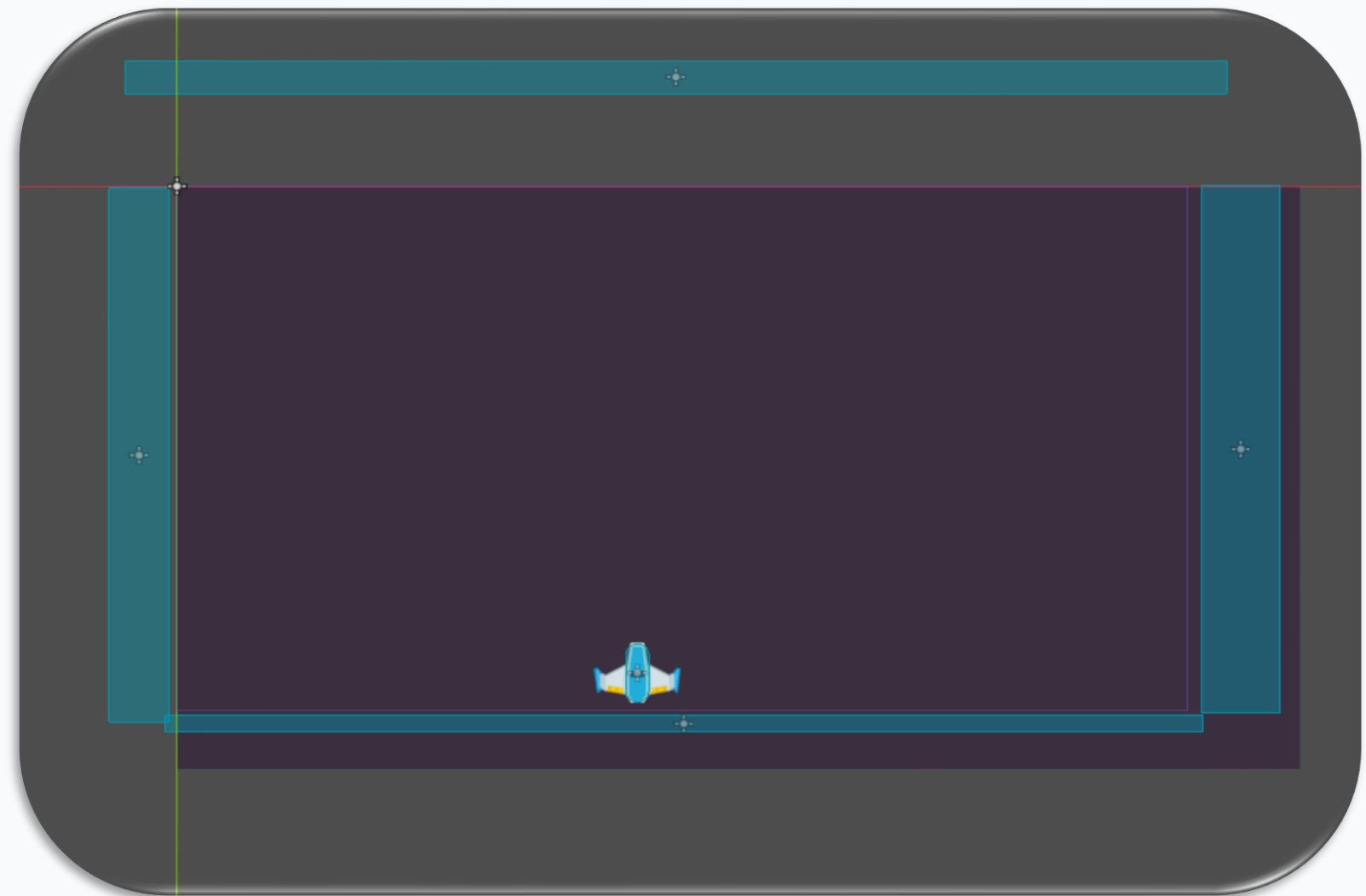


La scena Ship ora è figlia della scena Game e sarà trattata come nodo figlio.

# Demo live – Delimitiamo l'area di gioco



Vogliamo evitare che il giocatore possa andare fuori dalla zona dello schermo. Uno dei modi per delimitare il campo di gioco è quello di creare degli **StaticBody2D** con figlio un **CollisionShape2D** e posizionarli ai 4 bordi dello schermo



# Demo live - Asteroid Scene

---



Nodo di tipo Area2D dato che servirà solo per gestire le collisioni.

Come figli del nodo Laser, aggiungiamo:

- Sprite2D per disegnare l'immagine dell'asteroide.
- CollisionShape2D per gestire le collisioni con l'ambiente.

# Demo live - Definiamo Layers and Masks

Quando si lavora con nodi che possono collidere tra di loro, è necessario definire i layers e le masks.

Layer = Layer numerico sul quale viene definito il nodo attuale. Dove si trova il nodo?

Mask = Il o i layers che questo nodo vede. Con chi può interagire il nodo?

Un oggetto A può rilevare una collisione con un oggetto B, solo se il layer dell'oggetto B è nella lista delle mask dell'oggetto A

I layer/mask sono numerici, ma possiamo dare loro un nome per facilitare il lavoro in: Project -> Project Settings -> 2D Physics

**Ship – Layer 1**

The screenshot shows the 'Collision' settings for the Ship object. The 'Layer' dropdown is set to 1, and the 'Mask' dropdown is set to 3. The 'Mask' dropdown is open, showing a grid of 24 options (1-24) with 3 selected.

| Collision Layer and Mask for Ship |   |   |   |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
|-----------------------------------|---|---|---|----|----|----|----|----|----|----|----|--|--|--|--|--|--|--|--|--|--|--|--|
| Layer                             |   |   |   |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                 | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                 | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 |  |  |  |  |  |  |  |  |  |  |  |  |
| Mask                              |   |   |   |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                 | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                 | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 |  |  |  |  |  |  |  |  |  |  |  |  |

**Asteroid – Layer 3**

The screenshot shows the 'Collision' settings for the Asteroid object. The 'Layer' dropdown is set to 3, and the 'Mask' dropdown is set to 1. The 'Mask' dropdown is open, showing a grid of 28 options (1-28) with 1 selected.

| Collision Layer and Mask for Asteroid |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
|---------------------------------------|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|--|--|--|--|--|--|--|--|--|--|--|--|
| Layer                                 |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                     | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 | 25 | 26 | 27 | 28 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                     | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 | 29 | 30 | 31 | 32 |  |  |  |  |  |  |  |  |  |  |  |  |
| Mask                                  |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                     | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 | 25 | 26 | 27 | 28 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                     | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 | 29 | 30 | 31 | 32 |  |  |  |  |  |  |  |  |  |  |  |  |

**Laser – Layer 2**

The screenshot shows the 'Collision' settings for the Laser object. The 'Layer' dropdown is set to 2, and the 'Mask' dropdown is set to 3. The 'Mask' dropdown is open, showing a grid of 28 options (1-28) with 3 selected.

| Collision Layer and Mask for Laser |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
|------------------------------------|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|--|--|--|--|--|--|--|--|--|--|--|--|
| Layer                              |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                  | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 | 25 | 26 | 27 | 28 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                  | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 | 29 | 30 | 31 | 32 |  |  |  |  |  |  |  |  |  |  |  |  |
| Mask                               |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                  | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 | 25 | 26 | 27 | 28 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                  | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 | 29 | 30 | 31 | 32 |  |  |  |  |  |  |  |  |  |  |  |  |

**Wall – Layer 4**

The screenshot shows the 'Collision' settings for the Wall object. The 'Layer' dropdown is set to 4, and the 'Mask' dropdown is set to 1. The 'Mask' dropdown is open, showing a grid of 24 options (1-24) with 1 selected.

| Collision Layer and Mask for Wall |   |   |   |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
|-----------------------------------|---|---|---|----|----|----|----|----|----|----|----|--|--|--|--|--|--|--|--|--|--|--|--|
| Layer                             |   |   |   |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                 | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                 | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 |  |  |  |  |  |  |  |  |  |  |  |  |
| Mask                              |   |   |   |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |  |  |  |  |
| 1                                 | 2 | 3 | 4 | 9  | 10 | 11 | 12 | 17 | 18 | 19 | 20 |  |  |  |  |  |  |  |  |  |  |  |  |
| 5                                 | 6 | 7 | 8 | 13 | 14 | 15 | 16 | 21 | 22 | 23 | 24 |  |  |  |  |  |  |  |  |  |  |  |  |

# Demo live - Asteroid.gd

```
extends Area2D
```

```
@onready var sprite = $Sprite2D
```

```
var speed: int
```

```
var direction_x: float
```

```
var rotation_speed: int
```

```
var is_golden: bool = false
```

```
#Array contiene i percorsi logici alle immagini degli asteroidi
```

```
var sprites = ["res://sprites/asteroids/1.png",  
"res://sprites/asteroids/2.png", "res://sprites/asteroids/3.png",  
"res://sprites/asteroids/4.png", "res://sprites/asteroids/5.png",  
"res://sprites/asteroids/6.png", "res://sprites/asteroids/golden.png"]
```

```
func _ready() -> void:
```

```
    var spriteRandom = randi_range(0, sprites.size()-1)
```

```
    #Associo alla proprietà texture di sprite il path all'immagine
```

```
    sprite.texture = load(sprites[spriteRandom])
```

```
    #Seleziono randomicamente una posizione fuori schermo
```

```
    var width = get_viewport().get_visible_rect().size[0]
```

```
    var xPos = randi_range(0, width)
```

```
    var yPos = randi_range(-150, -50)
```

```
    position = Vector2(xPos, yPos)
```

```
#Speed/direction/rotation
```

```
    speed = randi_range(120, 300)
```

```
    direction_x = randf_range(-1, 1)
```

```
    rotation_speed = randi_range(40, 100)
```

```
    if spriteRandom == sprites.size()-1:
```

```
        is_golden = true
```

```
func _process(delta: float) -> void:
```

```
    position += Vector2(direction_x, 1.0) * speed * delta
```

```
    rotation_degrees += rotation_speed * delta
```

# Demo live - Game.gd

```
extends Node2D
```

```
@onready var asteroidTimer = $Timer
```

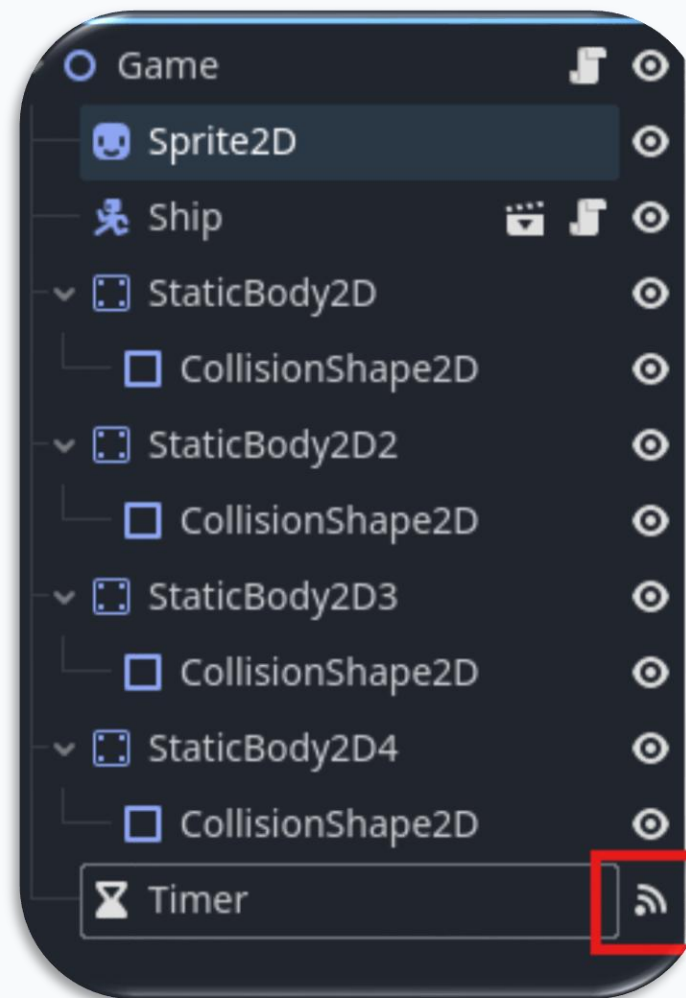
```
var asteroid = preload("res://scenes/asteroid.tscn")
var asteroidsGenerator = 5
```

```
func _ready() -> void:
    generate_asteroids()
```

```
#Generiamo gli asteroidi al primo avvio della scena e
#ogni volta che Timer lancia timeout
```

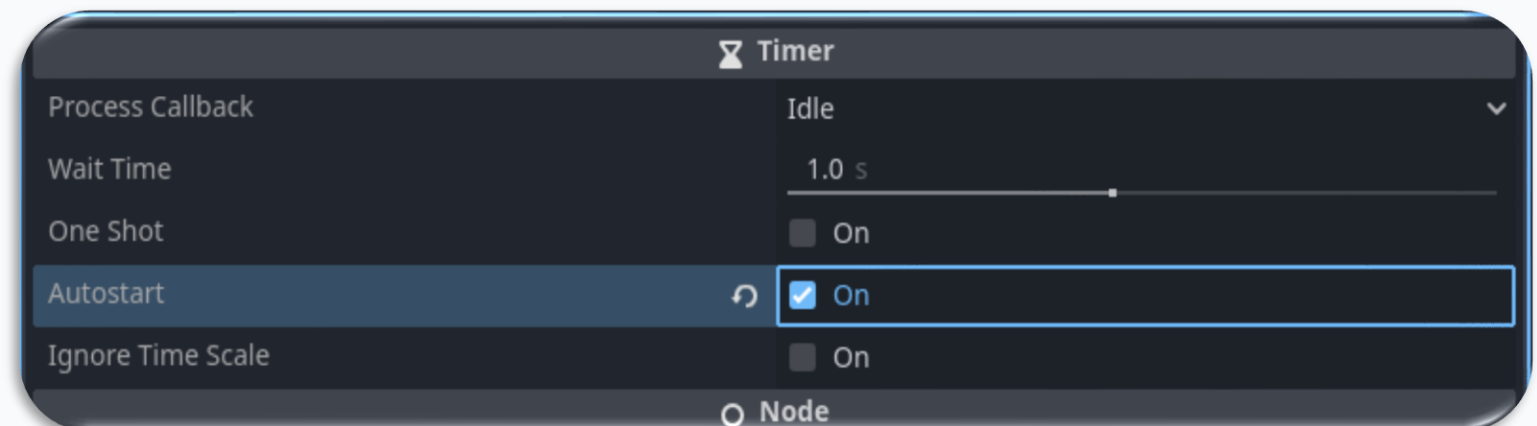
```
func generate_asteroids():
    for x in range(0, asteroidsGenerator):
        var asteroidScene = asteroid.instantiate()
        add_child(asteroidScene)
```

```
func _on_timer_timeout() -> void:
    generate_asteroids()
```

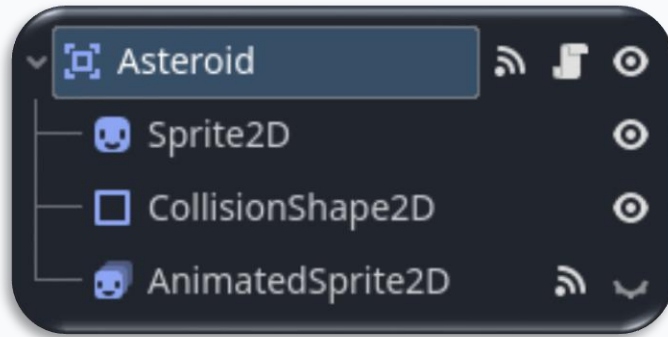


Il nodo Timer deve essere collegato tramite UI (o tramite codice) al nodo padre Game, in modo che allo scadere del tempo, il timer possa lanciare il segnale di timeout.

Lo configuriamo con Autostart=true, in modo che terminata l'esecuzione e chiamato il timeout(), il timer riparta da solo



# Demo live – Collisione con il laser e animazioni! Asteroid.gd



Andiamo a rilevare le collisioni tra gli asteroidi e il laser tramite il `_on_area_entered` (segnale base dei nodi `Area2D`)

Vogliamo anche riprodurre una semplice animazione quando il meteorite viene distrutto – Usiamo quindi il nodo `AnimatedSprite2D`, inizialmente non visibile.

Aggiungiamo:

```
@onready var animation = $AnimatedSprite2D
@onready var collisionShape = $CollisionShape2D
```

#--- Segnale da collegare del nodo padre Asteroid ---

#Per rilevare collisione con LASER che è un'Area2D

```
func _on_area_entered(area: Area2D) -> void:
```

```
    area.queue_free()
```

```
    destroy()
```

#Terminata l'animazione eliminiamo l'oggetto – Segnale da collegare al nodo `AnimatedSprite2D`

```
func _on_animated_sprite_2d_animation_finished() -> void:
```

```
    queue_free()
```

```
func destroy():
```

```
    sprite.visible = false
```

```
    animation.visible = true
```

```
    collisionShape.set_deferred("disabled", true)
```

```
    speed = 0
```

```
    rotation_speed = 0
```

```
    animation.play("explosion")
```

# Demo live – Collisione con la navicella! Asteroid.gd

---

## Asteroid.gd

```
#--- Segnale da collegare del nodo padre Asteroid ---  
#Per rilevare collisione con SHIP che è un CharacterBody2D  
func _on_body_entered(body: Node2D) -> void:  
    destroy()  
    if body.has_method("take_damage"):  
        body.take_damage()
```

Per rilevare collisioni con la ship, dobbiamo usare un'altro tipo di segnale che ricerca per il tipo CharacterBody2D – Il segnale **`_on_body_entered`**

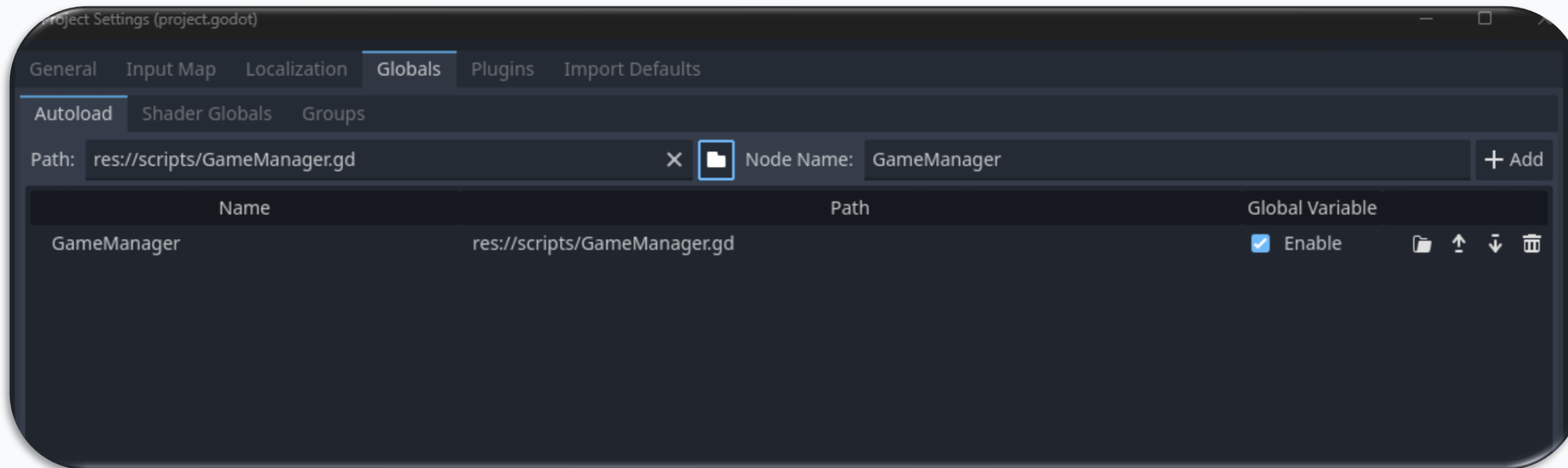


Ora dobbiamo solo creare la parte per gestire gli HP del giocatore tramite quando il metodo `take_damage` viene invocato

# Demo live – Autoload GameManager

Creiamo un Autoload o Global script:

1. Dentro la cartella scripts, creiamo un nuovo file chiamato GameManager.gd
2. In Project -> Project Settings -> Globals aggiungiamo il file come Autoload



3. Ora possiamo referenziare ogni variabile o metodo all'interno di GameManager da ogni classe del progetto

# Demo live - GameManager.gd

## GameManager.gd

extends Node

var hp = 3

var best\_score = 0

var current\_score = 0

func damage():

    hp -= 1

func increase\_score(is\_golden: bool):

    if is\_golden:

        current\_score+=5

    else:

        current\_score+=1

Nel nostro GameManager possiamo ora definire gli hp totali a disposizione del giocatore e una funzione per decrementarli.

Possiamo ora collegare la funzione **increase\_score** di **asteroid.gd** per tenere traccia dei punti:

## Asteroid.gd

func destroy():

    GameManager.increase\_score(is\_golden)

    sprite.visible = false

    animation.visible = true

    collisionShape.set\_deferred("disabled", true)

    speed = 0

    rotation\_speed = 0

    animation.play("explosion")

# Demo live - Animazioni con i Tween in Ship.gd

```
var is_immune = false
```

```
func take_damage():  
    if not is_immune:  
        GameManager.damage()  
        blink()
```

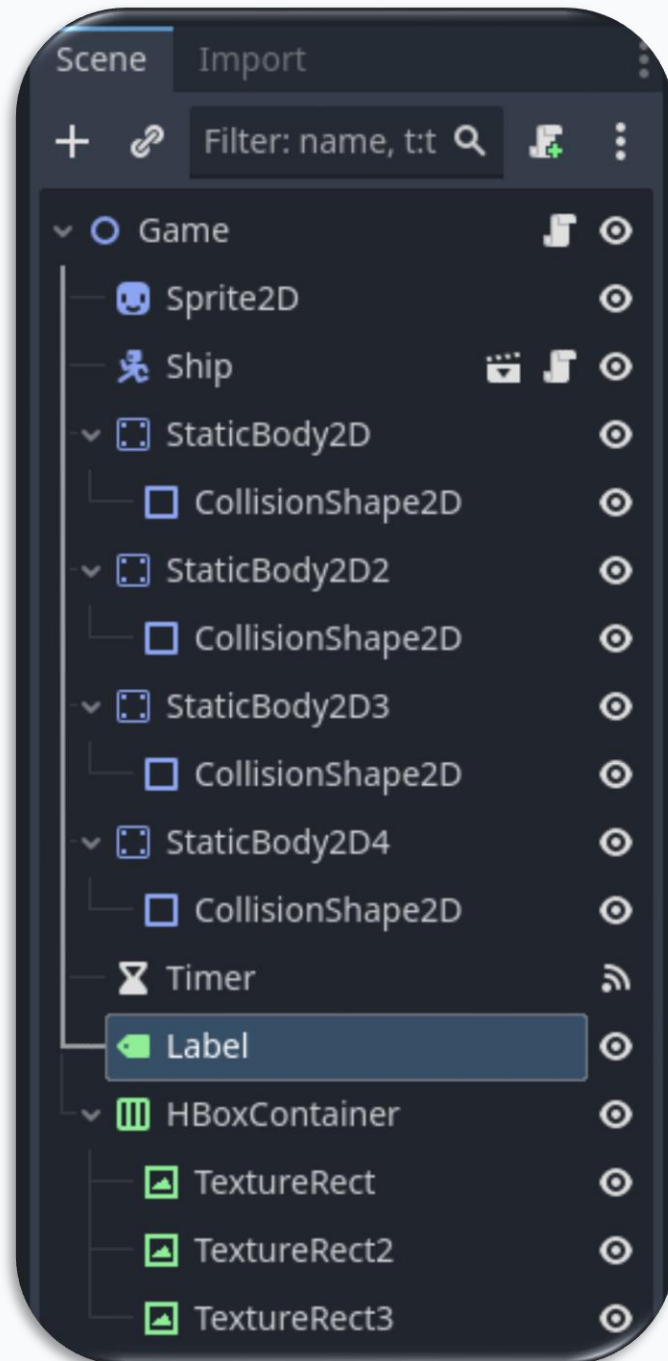
```
func blink():  
    is_immune = true  
    var tween = create_tween()  
    tween.set_loops()  
  
    tween.tween_property(self, "modulate:a", 0.0, 0.2)  
    tween.tween_property(self, "modulate:a", 1.0, 0.2)
```

```
await get_tree().create_timer(3.0).timeout  
tween.kill()  
modulate.a = 1.0  
is_immune = false
```

I tween sono degli oggetti che possono essere istanziati dinamicamente e offrono la possibilità di creare semplici animazioni via script.

In questo caso andiamo a ricreare l'effetto del blink che segnala che il giocatore è immune dai danni.

# Demo live – Aggiungiamo la UI in Game.gd



Il nodo Label contiene il punteggio attuale del giocatore che è salvato all'interno di GameManager, mentre il nodo HBoxContainer visualizza le vite rimanenti al giocatore

# Demo live - Colleghiamo i segnali tra GameManager.gd e Game.gd

## GameManager.gd

```
extends Node
```

```
var hp = 3
```

```
var best_score = 0
```

```
var current_score = 0
```

```
signal update_score(score: int)
```

```
signal update_life(life: int)
```

```
signal game_over()
```

```
func damage():
```

```
    hp -= 1
```

```
    update_life.emit(hp)
```

```
    if hp <= 0:
```

```
        if current_score > best_score:
```

```
            best_score = current_score
```

```
            game_over.emit()
```

```
func increase_score(is_golden: bool):
```

```
    if is_golden:
```

```
        current_score+=5
```

```
    else:
```

```
        current_score+=1
```

```
    update_score.emit(current_score)
```

## Game.gd

```
@onready var scoreLabel = $Label
```

```
@onready var lifeContainer = $HBoxContainer
```

```
func _ready() -> void:
```

```
    #Collego il segnale "update_score" di GameManager dentro il metodo _on_update_score
```

```
    GameManager.connect("update_score", _on_update_score)
```

```
    GameManager.connect("update_life", _on_update_life)
```

```
    GameManager.connect("game_over", _on_game_over)
```

```
    generate_asteroids()
```

```
func _on_update_score(score: int):
```

```
    scoreLabel.text = str(score)
```

```
func _on_update_life(life: int):
```

```
    for x in range(0,3):
```

```
        if life > 0:
```

```
            lifeContainer.get_children()[x].visible = true
```

```
        else:
```

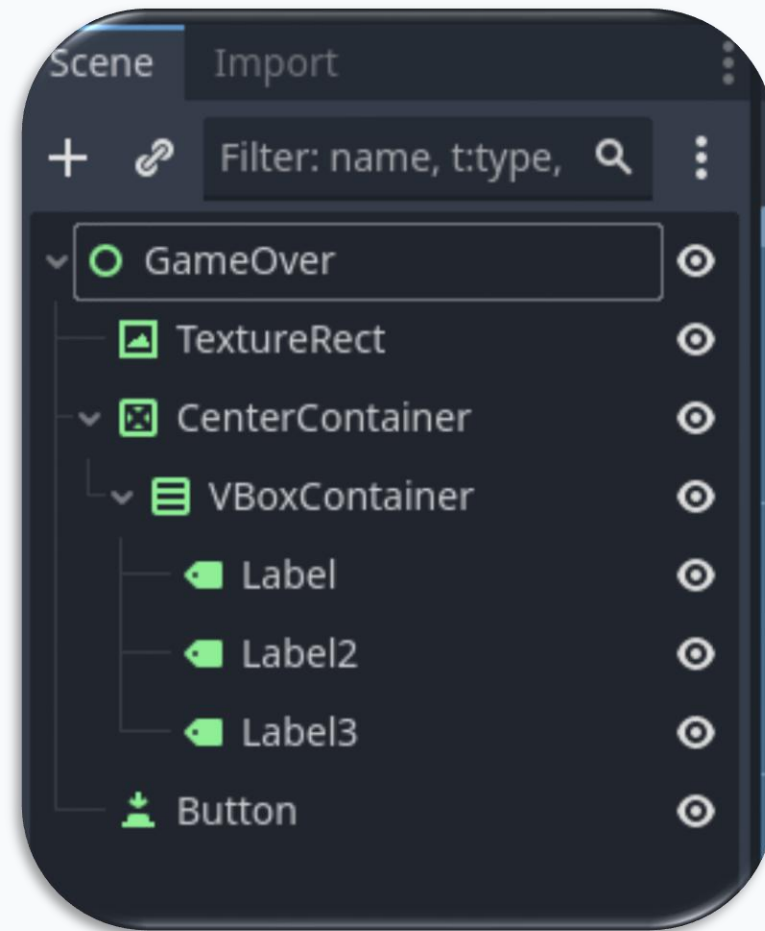
```
            lifeContainer.get_children()[x].visible = false
```

```
        life-=1
```

```
func _on_game_over():
```

```
    print("Game Over!")
```

## Demo live - Game Over?



Vogliamo che il giocatore possa rigiocare e tentare di battere il proprio record!  
Creiamo una nuova scena con soli nodi Control per mostrare i punteggi ed un bottone per far ripartire il gioco



# Demo live - gameOver.gd e GameManager.gd

## gameOver.gd

extends Control

@onready var score = \$CenterContainer/VBoxContainer/Label2

@onready var best\_score = \$CenterContainer/VBoxContainer/Label3

func \_ready() -> void:

    score.text += " " + str(GameManager.current\_score)

    best\_score.text += " " + str(GameManager.best\_score)

#Collegiamo il bottone al segnale pressed e resettiamo il

#GameManager prima di rilanciare la scena di gioco

func \_on\_button\_pressed() -> void:

    GameManager.reset()

    get\_tree().change\_scene\_to\_file("res://scenes/game.tscn")

## GameManager.gd

func reset():

    current\_score = 0

    hp = 3

# Demo live - Colleghiamo il segnale\_on\_game\_over in Game.gd

---

## Game.gd

```
#Call_deferred perchè non possiamo passare da una scena all'altra  
#prima che il motore non abbia finito il ciclo corrente di  
#calcoli  
func _on_game_over():  
    call_deferred("end_game")  
  
func end_game():  
    get_tree().change_scene_to_file("res://scenes/gameOver.tscn")
```

## Demo live – Q&A

---

